

Verantwoordingsdocument — ScreenFind

Leerlijn Frontend — Eindopdracht V3.6

Student	[vul naam in]
Inleverdatum	[vul in]
GitHub repository	https://github.com/iti101/screenfind

Inhoudsopgave

1. [Inleiding](#)
2. [Verantwoording keuzes](#)
3. [Mogelijke doorontwikkelingen](#)
4. [Bronnenlijst](#)
5. [Bijlage A — AI-prompts](#)

1. Inleiding

Dit document verantwoordt de technische keuzes in ScreenFind en reflecteert op beperkingen. De applicatie combineert TMDb (zoeken/details) met de NOVI Dynamic API (authenticatie + watchlist) in een React SPA met React Router en Context.

2. Verantwoording keuzes

Keuze 1 — React Context i.p.v. prop drilling voor auth

Keuze: `AuthProvider` + `useAuth` bewaart token, user en login/logout centraal.

Waarom niet prop drilling? Login-status is nodig in `Navbar`, `PrivateRoute`, `WatchlistButton` en pagina's. Props doorgeven zou elke tussenliggende component koppelen aan auth.

Reflectie: Eerdere aanpak (alleen lokale state in `LoginPage`) maakte private routes onmogelijk zonder duplicate state. Context maakt één bron van waarheid; nadeel is dat verkeerd gebruik van Context te brede re-renders kan geven. Daarom blijft Context beperkt tot auth, niet tot zoekresultaten.

Keuze 2 — Aparte API-modules (`tmdb.js` / `novi.js`) i.p.v. fetch in componenten

Keuze: Alle netwerkcalls zitten in `src/api/*` met genormaliseerde return-objecten.

Waarom? Componenten blijven gericht op UI-state (loading/error/success). Normalisatie (`movie.title` vs `tv.name`) voorkomt duplicatie in `Search` én `Detail`.

Reflectie: Eerst stond `fetch` in `SearchSect`; bij `DetailPage` ontstond copy-paste. Door helpers te extraheren volg ik DRY en kan ik errors eenduidig gooien (`throw new Error(...)`) die de UI toont.

Keuze 3 — React Router met dynamic + private routes

Keuze: Routes zoals `/details/:mediaType/:id` en `PrivateRoute` rond `/watchlist`.

Waarom niet scroll-snap single page? De opdracht eist routing, dynamic routes en private routes. Scroll-secties voldeden niet aan criterium 3.8.

Reflectie: Migratie van de landing-page kostte structuurwerk, maar maakte deep links en auth-guards mogelijk.

PrivateRoute met `Navigate + location.state.from` herstelt de gebruiker na login naar de oorspronkelijke bestemming.

Keuze 4 — JWT in `localStorage` + `jwt-decode` voor `userId`

Keuze: Token opslaan in `localStorage`, bij refresh decoderen en `/users/:id` ophalen.

Waarom? NOVI levert JWT; zonder decode ontbreekt het user-id voor profile-fetch.

Reflectie: `localStorage` is XSS-gevoeliger dan `httpOnly` cookies, maar binnen deze frontend-only opdracht is dit de gangbare NOVI-aanpak. Ik controleer `exp` bij restore om verlopen sessies op te ruimen.

Keuze 5 — Handgeschreven CSS + Flexbox, modulair per feature

Keuze: Geen Tailwind/Bootstrap; CSS-bestanden naast pagina's/componenten.

Waarom? Opdrachtverbod op out-of-the-box styling; Flexbox voor layouts (auth-card, detail layout, watchlist cards).

Reflectie: Zonder utility-framework is consistentie bewuster werk (gedeelde `.button` / `.status-message` in `App.css`). Dat dwingt hergebruik af en voorkomt "één megabestand".

3. Mogelijke doorontwikkelingen

1. **Paginerig / infinite scroll op zoekresultaten** — Nu alleen de eerste TMDB-pagina. Limitatie: lange queries tonen niet alles. Volgende versie: page-parameter + "Load more".
2. **Watchlist-filters en sortering** — Statusfilter bestaat per item, niet als globale lijstfilter. Wenselijk: filter op status + zoeken binnen de lijst.
3. **Offline / caching** — Geen cache; herhaalde detailrequests. Verbetering: React Query of lokale cache met TTL.
4. **E-mailverificatie / wachtwoord reset** — NOVI-onderwijs-API biedt dit beperkt; accounts zijn kwetsbaar voor typfouten. Bij eigen backend of rijkere API toevoegen.
5. **Sociale features** — Geen reviews/delen. Een `reviews`-collectie in NOVI (zoals in vergelijkbare eindopdrachten) zou kernfunctionaliteit 4 kunnen uitbreiden zonder TMDB te overvragen.

Deze punten zijn bewust buiten scope gehouden om de vier kernflows stabiel en beoordeelbaar te houden.

4. Bronnenlijst

NOVI Hogeschool. (z.d.). *Eindopdracht leerlijn Frontend (V3.6)*.

The Movie Database. (z.d.). *API documentation*. <https://developer.themoviedb.org/docs>

NOVI. (z.d.). *NOVI Dynamic API documentation*. <https://novi-backend-api-wgsgz.ondigitaalocean.app/documentation/1-Overview>

React. (z.d.). *Passing data deeply with context*. <https://react.dev/learn/passing-data-deeply-with-context>

Remix / React Router. (z.d.). *React Router documentation*. <https://reactrouter.com/>

Cursor. (2026). *AI coding assistant* [Large language model]. <https://cursor.com>

Bijlage A — AI-prompts

1. "Go through Eindopdracht Frontend V3.6 PDF and list all requirements and steps to succeed."
2. "Implement the Eindopdracht plan: React Router, NOVI auth Context, private routes, TMDB search/details, watchlist CRUD."
3. "Write a verantwoordingsdocument with 5 technical choices and 5 functional limitations in APA style."

4. (vul aan met prompts die jij zelf hebt gebruikt)